

Java Streams API – P2

Stream creation; intermediate & terminal operations; Collectors
API; parallel streams awareness

Rahul Guha || SME || Java FS Training || HCL
2. Modern Java (8–21)

Terminal Operations

- Terminal operations produce a final result or side effect.
- Once a terminal operation executes, the Stream is **consumed** and cannot normally be reused.

Operation	Purpose
<code>forEach()</code>	Process each element
<code>collect()</code>	Convert into collection/result
<code>count()</code>	Count elements
<code>reduce()</code>	Combine elements
<code>min()</code>	Find minimum
<code>max()</code>	Find maximum
<code>findFirst()</code>	First element
<code>findAny()</code>	Any element
<code>anyMatch()</code>	Check if any matches
<code>allMatch()</code>	Check if all match
<code>noneMatch()</code>	Check if none match

Terminal Operations

- The `java.util.stream.Collectors` class is a utility class introduced in Java 8 that provides a rich set of pre-built, static factory methods to perform mutable reduction operations on elements of a Stream. It works hand-in-hand with the `.collect()` terminal operation of the Stream API to gather processed data back into data structures like collections, maps, strings, or single calculated values.

Common collectors:

- `Collectors.toList()`
- `Collectors.toSet()`
- `Collectors.toMap()`
- `Collectors.joining()`
- `Collectors.groupingBy()`
- `Collectors.partitioningBy()`
- `Collectors.counting()`
- `Collectors.summingInt()`
- `Collectors.averagingInt()`

Terminal Operations

- **Map Vs Filter**

map()	filter()
Transforms elements	Selects elements
Usually changes element values/types	Keeps original element type
One input → one output	Input may be removed
Returns Stream	Returns Stream

Important Stream Rules :

- **Rule 1 :** A Stream does not store data.
- **Rule 2 :** A Stream does not normally modify the source collection.
- **Rule 3 :** Intermediate operations are lazy.
- **Rule 4 :** A terminal operation starts stream processing.
- **Rule 5 :** A Stream should normally not be reused after a terminal operation.
- **Rule 6:** Streams can be sequential or parallel.
- **Rule 7:** Parallel streams require careful consideration of thread safety and ordering.
- **Rule 8 :** Avoid unnecessary side effects inside stream pipelines.

Stream API Quick Reference

CREATION

- |
- |—— collection.stream()
- |—— collection.parallelStream()
- |—— Arrays.stream(array)
- |—— Stream.of(...)
- |—— Stream.empty()
- |—— Stream.generate(...)
- |—— Stream.iterate(...)

INTERMEDIATE

- |
- |—— filter()
- |—— map()
- |—— flatMap()
- |—— distinct()
- |—— sorted()
- |—— limit()
- |—— skip()
- |—— peek()

TERMINAL

- |
- |—— forEach()
- |—— collect()
- |—— count()
- |—— reduce()
- |—— min()
- |—— max()
- |—— findFirst()
- |—— findAny()
- |—— anyMatch()
- |—— allMatch()
- |—— noneMatch()

Stream API Quick Reference

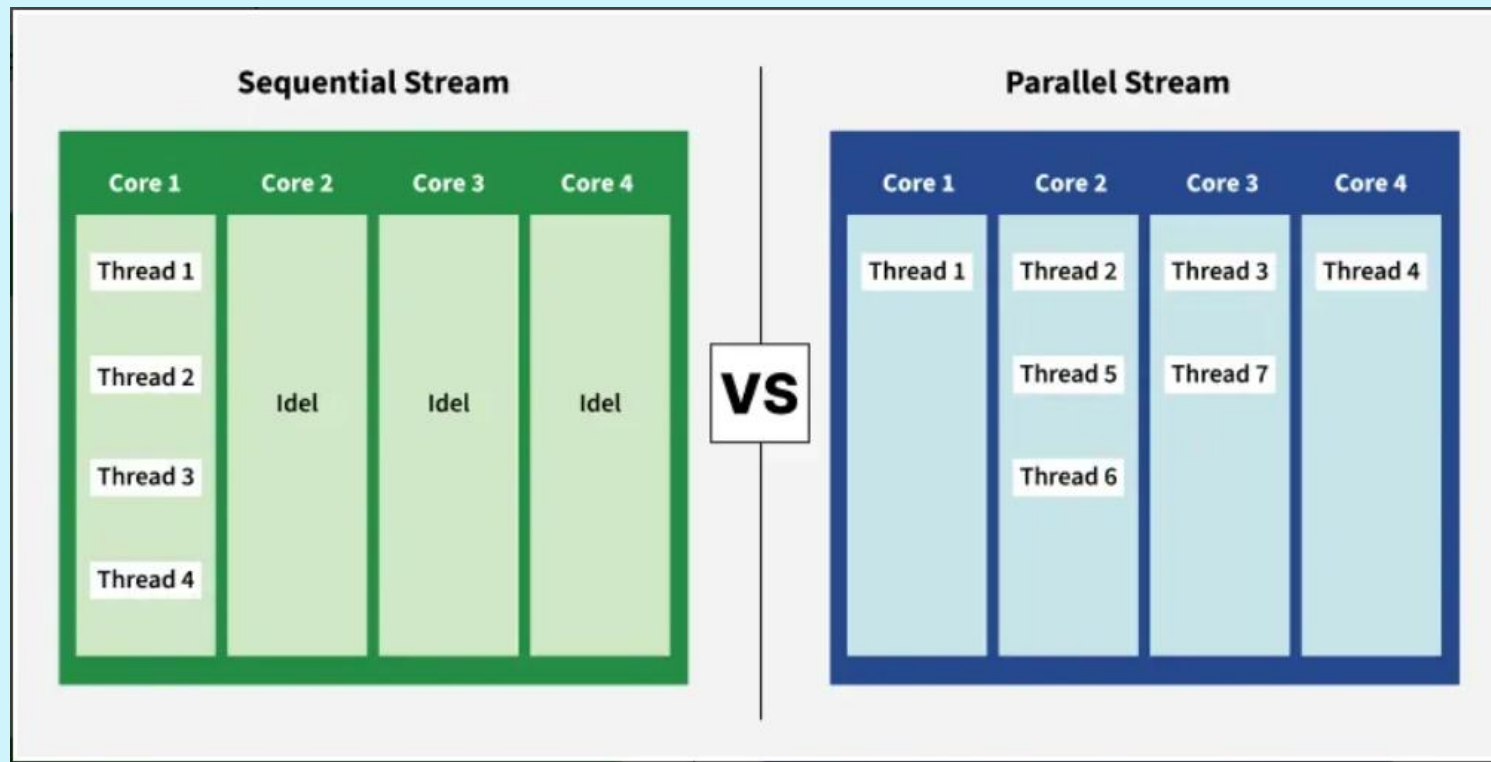
Java Parallel Streams is a feature of Java 8 and higher, meant for utilizing multiple cores of the processor. Normally any Java code has one stream of processing, where it is executed sequentially. Whereas by using parallel streams, we can divide the code into multiple streams that are executed in parallel on separate cores and the final result is the combination of the individual outcomes. The order of execution, however, is not under our control.

Therefore, it is advisable to use parallel streams in cases where no matter what is the order of execution, the result is unaffected and the state of one element does not affect the other as well as the source of the data also remains unaffected. Parallel streams are best used when the order doesn't matter, elements don't depend on each other, and data remains unchanged.

Parallel streams enable large-scale data processing tasks to be handled more efficiently by utilizing the full power of multi-core processors.

Stream API Quick Reference

The below image demonstrates the difference between Sequential Streams and Parallel Streams:



Sequential vs Parallel Streams

The main difference between Sequential and Parallel Stream is listed below:

Sequential Streams: Process elements in a sequential manner, one element at a time

Parallel Streams: Process elements in parallel, utilizing multiple CPU cores.

Stream API Quick Reference

Why Parallel Streams?

Parallel Streams improve performance by utilizing multiple cores, but they aren't always the best choice. In cases where the task requires a specific order of execution, sequential streams are more reliable, even if it means sacrificing performance. The performance gain from parallel streams is significant only for large-scale or computationally intensive programs. For smaller tasks, the difference may be negligible. Use parallel streams primarily when the sequential stream behaves poorly.

How to Create and Use Parallel Streams

There are two main ways to create parallel streams in Java:

1. Using the `parallel()` Method

The `parallel()` method is used on an existing sequential stream to convert it into a parallel stream. This method returns a parallel stream, which can then be processed using various stream operations.

Stream API Quick Reference

Why Parallel Streams?

Parallel Streams improve performance by utilizing multiple cores, but they aren't always the best choice. In cases where the task requires a specific order of execution, sequential streams are more reliable, even if it means sacrificing performance. The performance gain from parallel streams is significant only for large-scale or computationally intensive programs. For smaller tasks, the difference may be negligible. Use parallel streams primarily when the sequential stream behaves poorly.

How to Create and Use Parallel Streams

There are two main ways to create parallel streams in Java:

1. Using the `parallel()` Method

The `parallel()` method is used on an existing sequential stream to convert it into a parallel stream. This method returns a parallel stream, which can then be processed using various stream operations.

Stream API Quick Reference

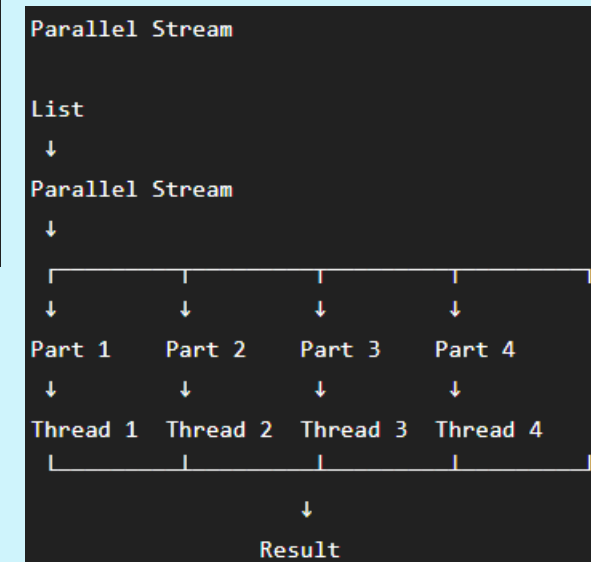
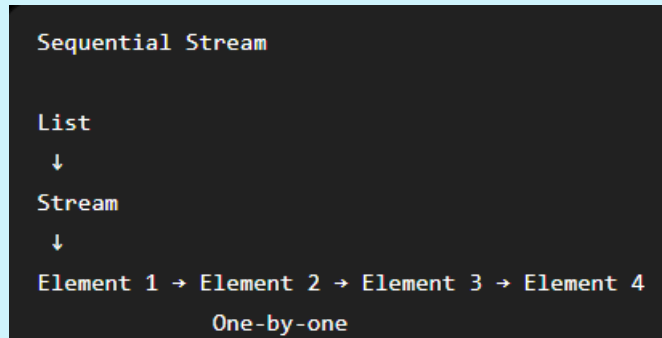
2. Using the `parallelStream()` Method

The `parallelStream()` method in Java is used to create a `parallelstream` from a `Collection`. We can directly call `parallelStream()` on `Collections` (such as `List`, `set`) to obtain parallel stream.

- Example: This example demonstrates reading lines from a file and processing them in parallel using a parallel stream.

Why Use Parallel Streams?

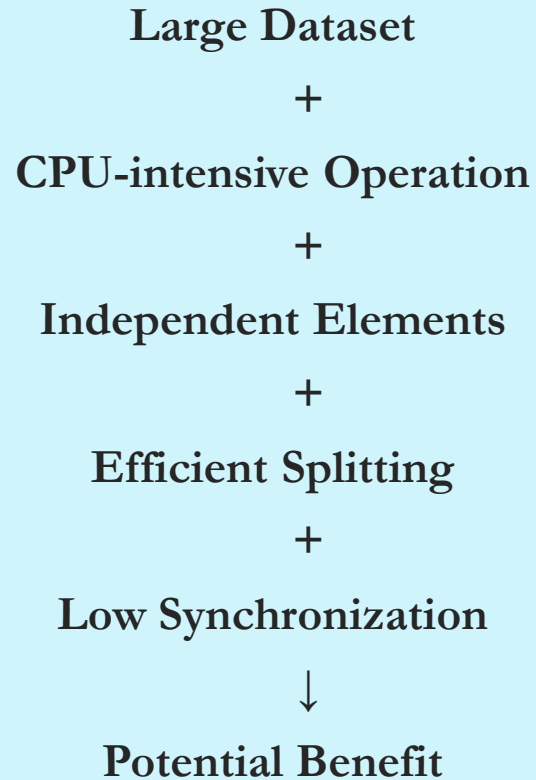
- Parallel streams can be useful when:
- Dataset is sufficiently large
- Processing is CPU-intensive
- Operations are independent
- The source can be efficiently split
- The workload benefits from multiple CPU cores



Stream API Quick Reference

- **When Parallel Streams May Be Useful**

A parallel stream may be worth considering when:



Example workloads:

- Large numerical calculations
- CPU-intensive transformations
- Large independent data-processing operations
- Aggregations that can be efficiently reduced

When to Avoid Parallel Streams

1. Small dataset
2. I/O-heavy work
3. Shared mutable state
4. Strict ordering is required
5. Operations have side effects

Sequential vs Parallel — Complete Comparison

Feature	Sequential Stream	Parallel Stream
Creation	<code>stream()</code>	<code>parallelStream()</code>
Processing	Sequential	Concurrent processing
Multiple threads	Usually no	Potentially yes
Ordering with <code>forEach()</code>	Encounter order generally maintained	Not guaranteed
<code>forEachOrdered()</code>	Ordered	Ordered
Overhead	Lower	Higher
Small data	Often preferable	Often unnecessary
Large CPU-bound workload	May benefit less	May potentially benefit
Shared state	Simpler	Requires more care
Debugging	Easier	More difficult
Performance	More predictable	Workload-dependent